F1 Podium Prediction: Milestone 1 Report (Team 47)

1. Introduction

We predict, for every driver in a Formula 1 race, whether they finish on the podium (top 3). Our final model is a logistic regression with a test PR-AUC of 0.712 on the 2022 to 2024 seasons.

- **Target:** `podium = 1` if `positionOrder <= 3`, otherwise 0.
- **Cutoff:** a feature may only use information known after qualifying and before the race starts: this race's grid and qualifying, plus results and standings of earlier races.
- **Data:** the Kaggle Formula 1 World Championship dataset (1950 to 2024), 14 linked CSV tables.
- **Links:** Kaggle notebook: kaggle.com/code/mahmoudhatemzaky/f1-podium-prediction-milestone-1-team-47. GitHub repository: github.com/mahmoud-hatem-zaky/team47-acl-prodium-prediction.

2. Data cleaning and EDA

The data was mostly clean already: the main work was turning missing values into real `NaN` and removing 91 duplicate driver-race rows. The before and after evidence (missing values per table and per column, row counts and column types) is in Appendix A.

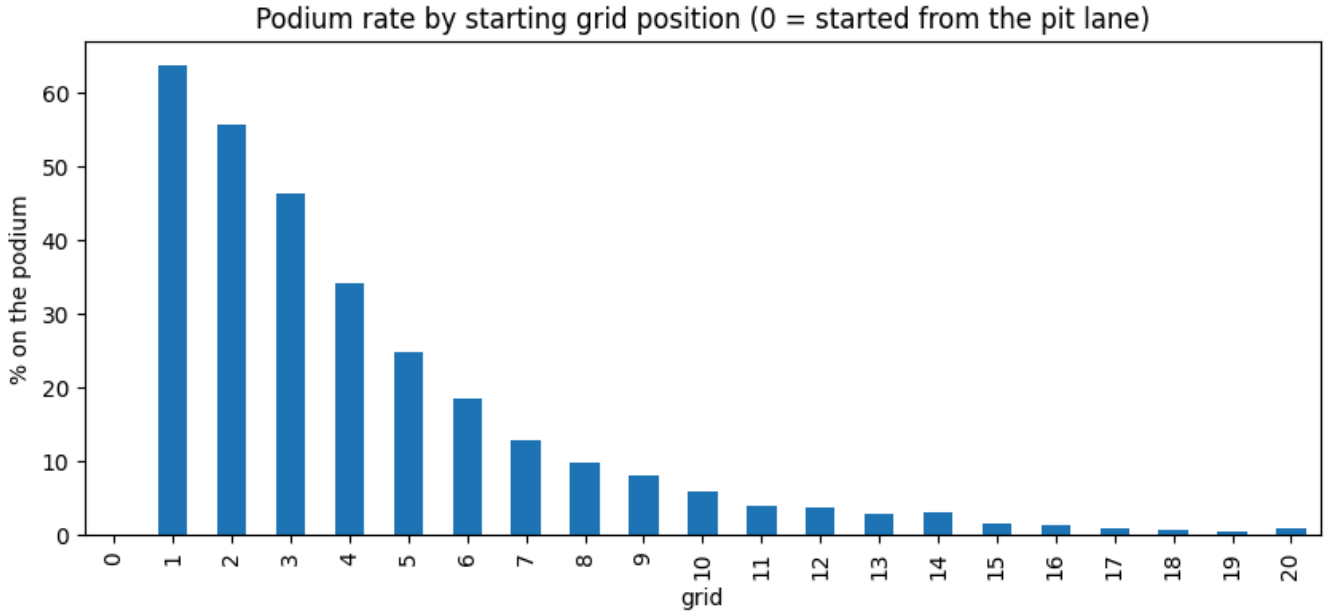
Cleaning steps

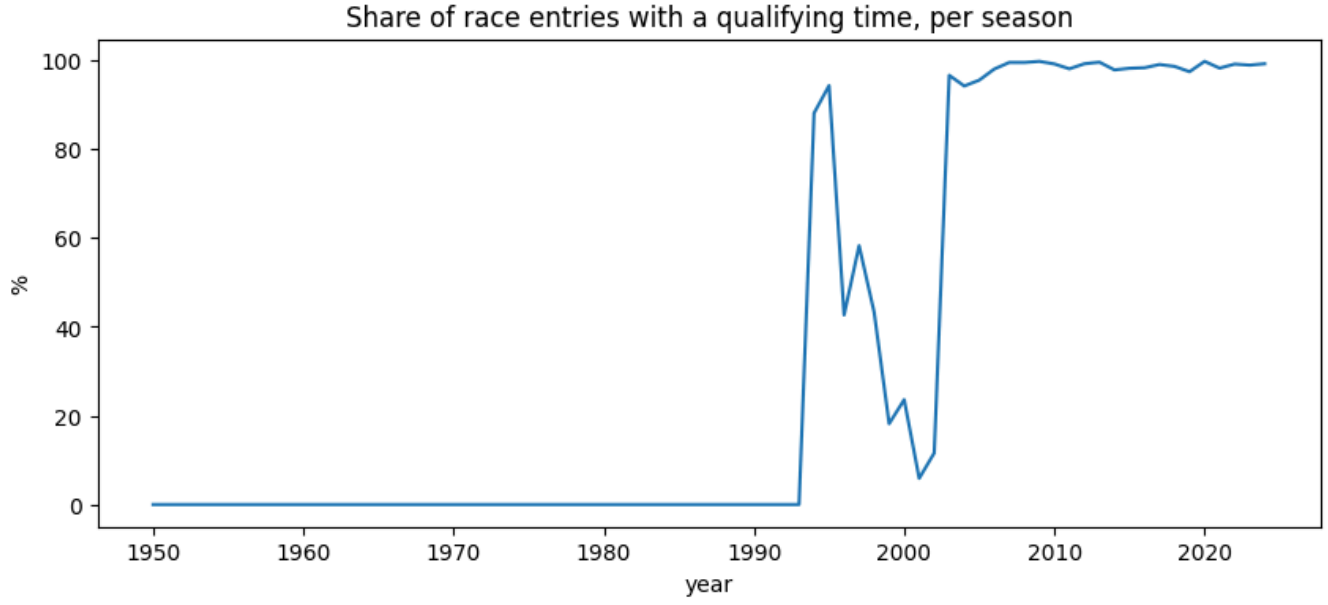
1. Missing values are written as the text `\N` in this dataset. We replaced every `\N` with `NaN` and converted the columns that only held numbers back to numeric types.
2. Dates (`races.date`, `drivers.dob`) were converted to real dates, and qualifying lap times like `1:23.456` were converted to seconds.
3. **Keys:** we checked the primary key of all 14 tables, including the composite keys (raceId, driverId, lap) for lap_times and (raceId, driverId, stop) for pit_stops, which have no ID column. Every key is unique and never missing. We also checked 23 foreign keys across races, results, qualifying, both standings tables, constructor results, sprint results, lap times and pit stops, and every value points to an existing row.
4. **Duplicates:** our model needs one row per driver per race. In early F1 a driver could drive more than one car in the same race (shared drives), which gave 176 duplicate rows, mostly 1950 to 1964 and one in 1978. We keep the row of the car the driver started in

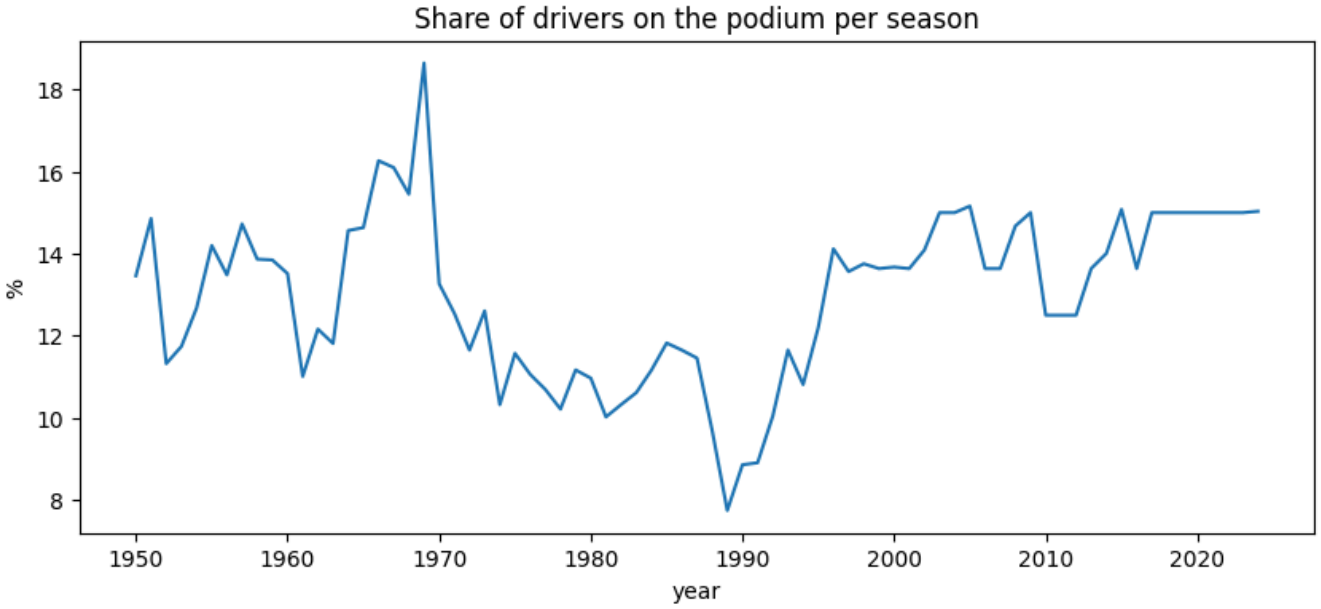
(grid above 0), because grid and qualifying features must describe that car. Choosing the row with the best finish would use the race result, which is leakage. This removed 91 rows: `results` went from 26,759 to 26,668 rows.

EDA findings

- **Grid position is the strongest signal.** About 64 % of pole sitters reach the podium, 46 % from 3rd, 25 % from 5th and under 10 % from 9th onwards. (Figure 1)
- **Qualifying data is incomplete in older seasons.** There is none before 1994, it is patchy from 1996 to 2002 and almost complete from 2003. (Figure 2)
- **The positive class is rare.** The podium share is 8 % to 19 % per season and about 15 % in recent seasons, so we use PR-AUC and F1 instead of accuracy. (Figure 3)





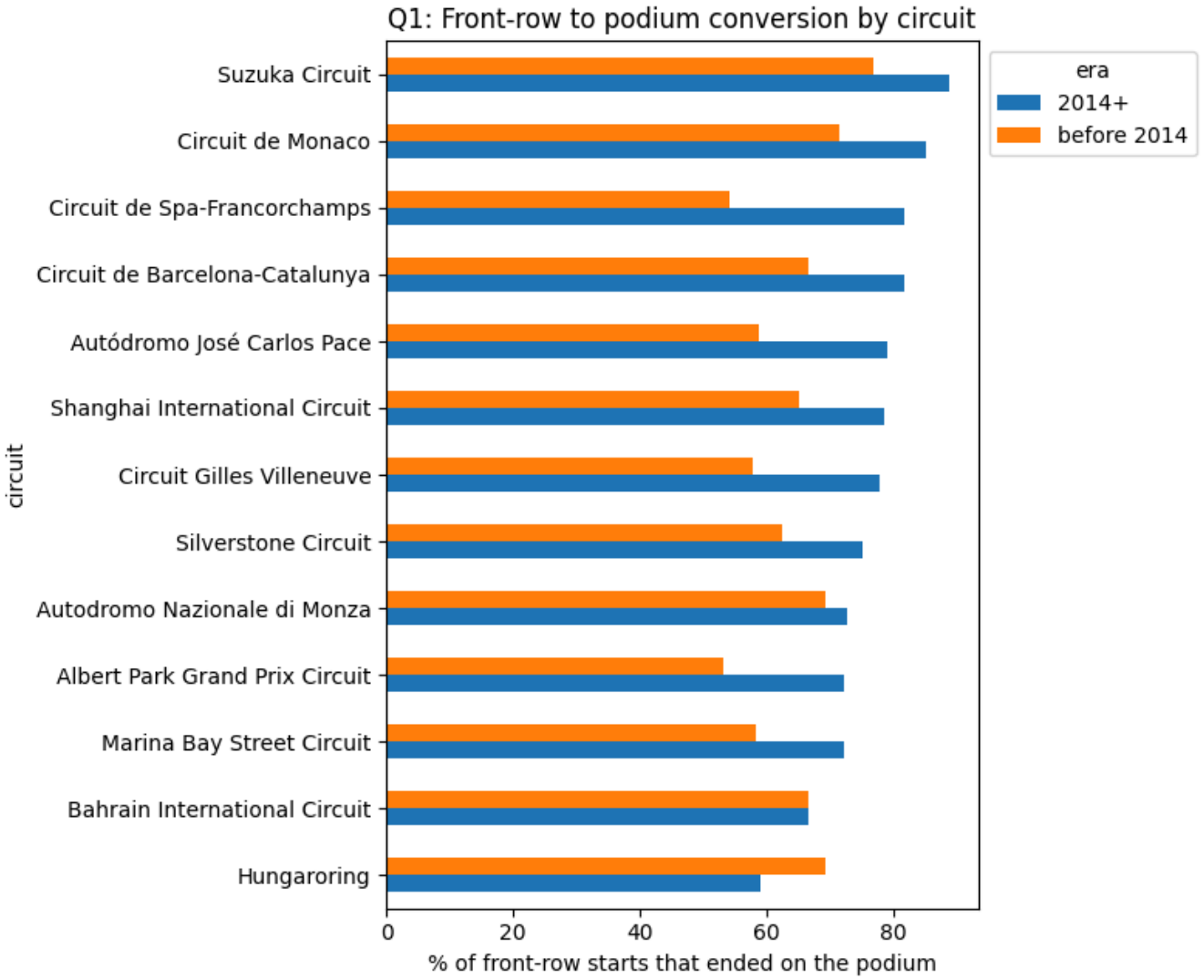


3. Data-engineering questions

Q1: Which circuits turn a front-row start into a podium most often? Did that change after 2014?

Front-row conversion went up after 2014 at 11 of 13 circuits; Suzuka and Monaco stay on top in both eras.

- **Method:** front row = starting 1st or 2nd; conversion = share of front-row starters who finish top 3. Tables: `results`, `races`, `circuits`, `qualifying`. Only rows with qualifying data (from 1994), and only circuits with enough starts in both eras.
- **Before 2014:** Suzuka (77 %), Monaco (71 %), Monza and the Hungaroring (69 %).
- **From 2014:** Suzuka (89 %), Monaco (85 %), Spa and Barcelona (82 %). Spa improved the most (+28 points); only the Hungaroring dropped (-10 points).
- The rank agreement between the eras is only 0.23 (Spearman). Defining the front row by qualifying instead of grid gives almost the same ranking (0.88), even though the two disagree in 74 rows because of grid penalties.
- **Limit:** each rate is based on a few dozen starts, so a change at a single circuit can be noise.



Q2: Do drivers podium more often at home, for the same starting position?

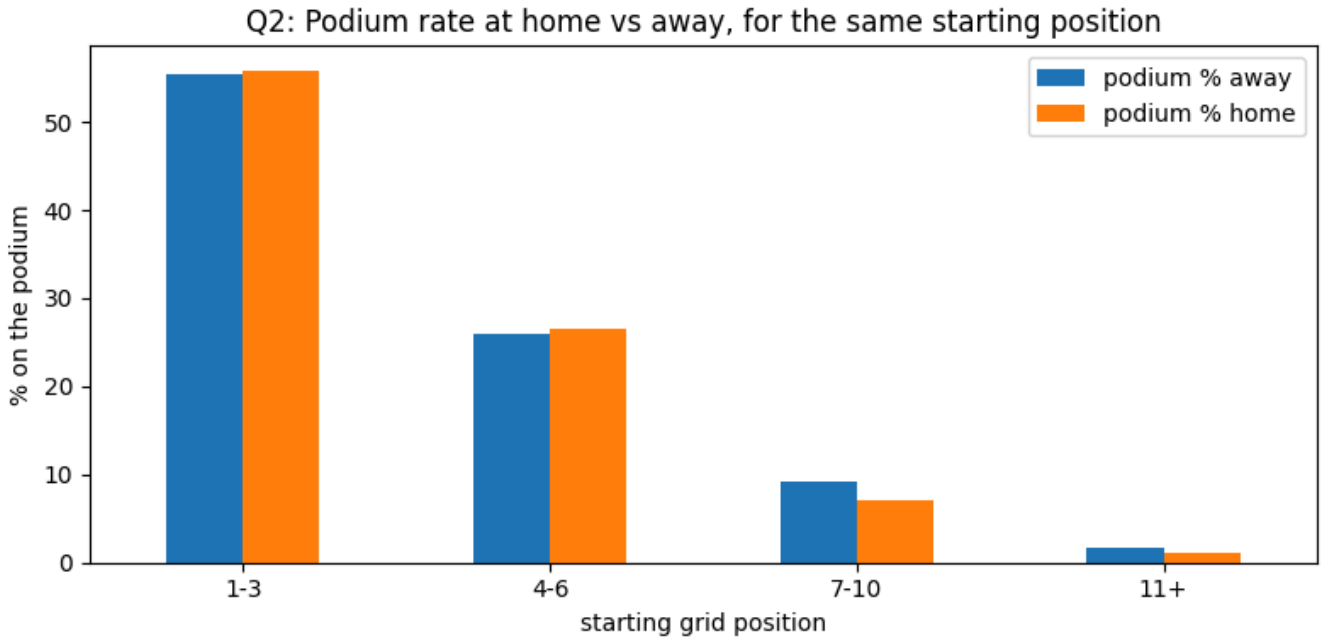
We observed no clear home advantage after comparing drivers within grid-position bands: home drivers podium 0.6 percentage points less often than away drivers.

- **Method:** home race = driver nationality matches the circuit country (hand-written mapping). Tables: `results`, `drivers`, `races`, `circuits`. Indianapolis 500 removed (mostly American drivers who raced nowhere else). 24,652 rows, 1,942 of them home races.
- Ignoring the grid, home drivers podium 2.1 points less often. Comparing inside grid bands (1-3, 4-6, 7-10, 11+), the gap shrinks to -0.6 points, and every band is within about 2 points.

Grid band	Podium % away	Podium % home	Difference
1-3	55.4	55.8	+0.5
4-6	26.0	26.5	+0.5

Grid band	Podium % away	Podium % home	Difference
7-10	9.2	7.1	-2.1
11+	1.7	1.0	-0.6

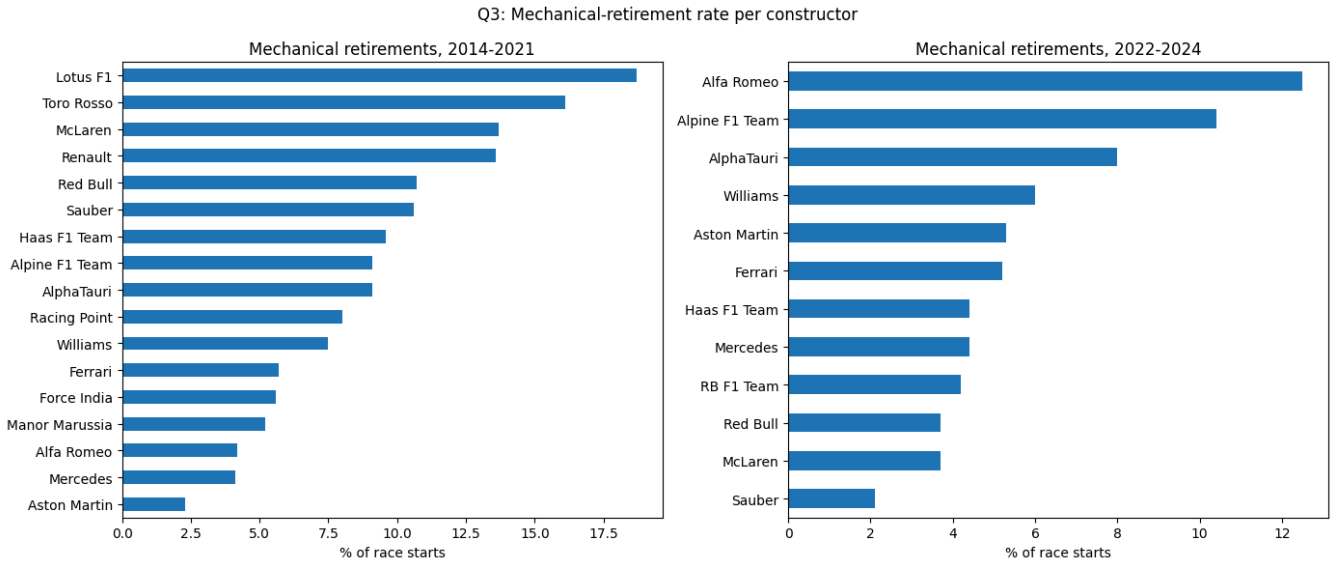
- **Limit:** nationality is not always "home", dual nationalities have no home race, and the front bands have only about 230 home rows each.



Q3: Which constructors have the highest mechanical-retirement rate, 2014-2021 vs 2022-2024?

Lotus F1 (18.7 %) led in 2014-2021 and Alfa Romeo (12.5 %) in 2022-2024; across all teams the rate fell from 9.4 % to 5.8 % of starts.

- **Method:** a mechanical retirement is a start that ended because a car part failed (engine, gearbox, brakes and so on). Crashes, disqualifications and unexplained "Retired" do not count. Tables: `results`, `status`, `races`, `constructors`. Rate = mechanical retirements / starts; teams with fewer than 40 starts in an era are left out.
- **2014-2021:** Lotus F1 (18.7 %), Toro Rosso (16.1 %), McLaren (13.7 %).
- **2022-2024:** Alfa Romeo (12.5 %), Alpine (10.4 %), AlphaTauri (8.0 %).
- McLaren (13.7 % to 3.7 %) and Red Bull (10.7 % to 3.7 %) improved the most; Alfa Romeo got worse (4.2 % to 12.5 %).
- **Limit:** renamed teams count as separate teams (for example Toro Rosso, AlphaTauri, RB).



4. Modeling table, features and split

The modeling table has 24,586 rows (one per driver per race), 15 pre-race features in 4 groups, and a podium rate of 13.6 %.

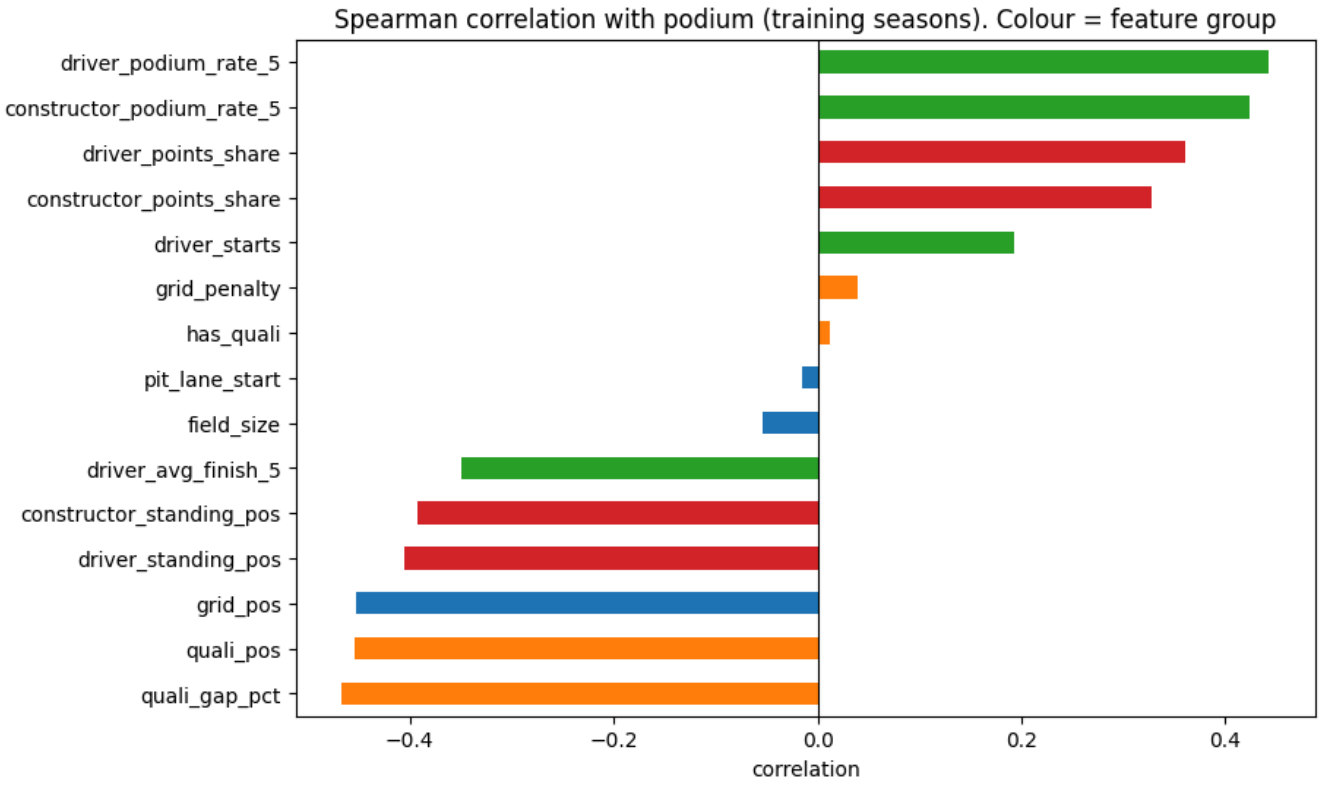
Rows removed: the Indianapolis 500 (1950 to 1960; 381 entries, drivers who rarely raced elsewhere) and entries that never started (`positionText` F or W; 1,701 rows, 0 podiums). Pit-lane starters (`grid = 0`) are kept because they do race.

Selected features and why each is known before the race

Group	Feature	Meaning	Why it is valid at the cutoff
grid	`grid_pos`	Starting position after penalties (pit lane = last place)	The grid is fixed before the start
grid	`pit_lane_start`	1 if starting from the pit lane	Same
grid	`field_size`	Number of cars starting	Same
qualifying	`quali_pos`	Qualifying position (grid used when no qualifying data)	Qualifying ends before the race
qualifying	`has_quali`	1 if real qualifying data exists	Same
qualifying	`quali_gap_pct`	% slower than pole, capped at 7 %	Same

Group	Feature	Meaning	Why it is valid at the cutoff
qualifying	`grid_penalty`	`grid_pos - quali_pos`	Both known before the start
form	`driver_podium_rate_5`	Podium share in the driver's previous 5 starts	Only earlier races (shifted window)
form	`driver_avg_finish_5`	Average finish in the previous 5 starts	Same
form	`driver_starts`	Number of earlier starts	Same
form	`constructor_podium_rate_5`	Team podiums per car over its previous 5 races	Same
standings	`driver_standing_pos`	Championship position before this race	Standings after the previous round
standings	`driver_points_share`	Driver points / leader points before this race	Same
standings	`constructor_standing_pos`	Constructor position before this race	Same
standings	`constructor_points_share`	Constructor points / leading constructor points	Same

Evidence for the choices (notebook 3.5): grid and qualifying have the strongest correlation with podium; form and standings are almost as strong; drivers who qualified top 6 but lost 1 to 4 grid places podium only about 9 % of the time against 45 % for those who kept their place, which is why both `quali_pos` and `grid_penalty` are kept. Points share is used instead of raw points because the points system changed many times. (Figure 7)



Leakage test: for 30 random races we scrambled the results and standings of that race and every later race, rebuilt the table, and checked that the features of that race did not change. It passes, and later races did change, so the test can detect leakage.

Split (by season)

Split	Seasons	Races	Rows	Podium rate
Train	1950-2019	1,007	22,459	13.5 %
Validation	2020-2021	39	775	15.1 %
Test	2022-2024	68	1,352	15.1 %

We split by season, not at random, because the model predicts the future from the past, form features chain neighbouring races together, all drivers of one race must stay in the same split, and 2022 brought new car rules, so the test checks a new era. Thresholds and hyperparameters were chosen on validation only.

Preprocessing: median imputation and standard scaling inside a scikit-learn `Pipeline`, fitted on training rows only. No one-hot encoding of driver or team IDs, because new drivers and renamed teams appear in the test seasons. The effect of imputation and scaling on the feature distributions is shown in Appendix B.

Preprocessing-order experiments (logistic regression, validation PR-AUC)

Experiment	Validation PR-AUC	Change vs baseline
Baseline: impute then scale, fitted on train	0.7157	0.0000
Imputer and scaler fitted on all seasons (leaky)	0.7160	+0.0003
Scale then impute	0.7161	+0.0004
No grid fallback, median imputation only	0.7189	+0.0032
Train from 1970 instead of 1950	0.7203	+0.0046

All observed differences are small (at most 0.005), and resampling the validation races moves this PR-AUC by about 0.056, so we do not treat them as meaningful and chose by correctness: fit on train only, impute then scale, keep the grid fallback (a pole sitter gets `quali_pos = 1`, not the median 11), and train from 1950.

5. Models and results

Logistic regression was selected because it achieved the highest validation PR-AUC (0.716) among the candidates, and it scored 0.712 on the unseen test set; the other models offered different precision–recall tradeoffs.

Metrics. Primary: **PR-AUC** (average precision), because podiums are the minority class and what matters is ranking real podium finishers above the many non-podium rows. Secondary: **F1** at the chosen threshold, which measures the yes/no predictions. We also report ROC-AUC, precision and recall. Each model's threshold maximizes F1 on validation.

Attempts (each in its own notebook cell, all with seed 47)

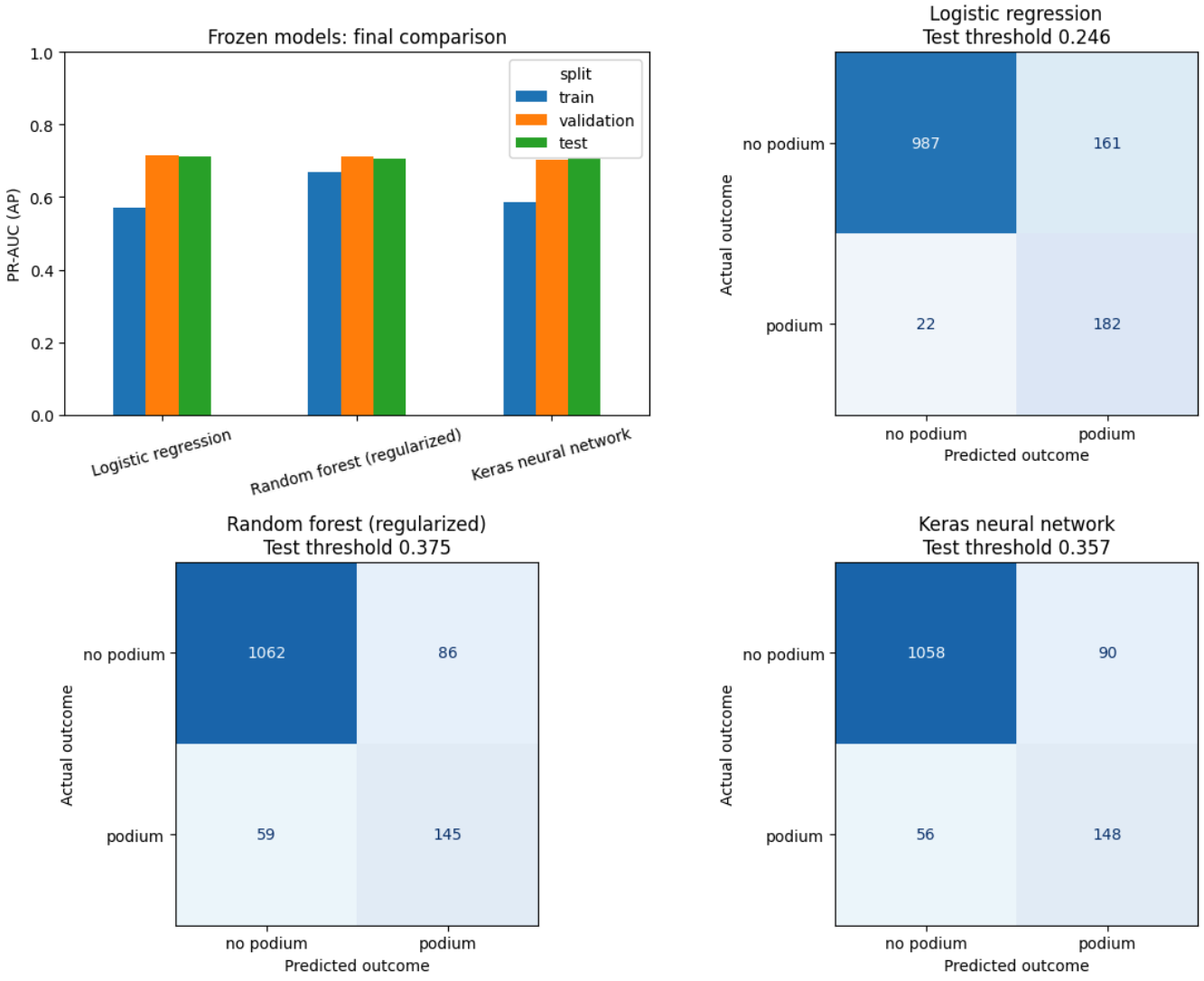
1. **Logistic regression (baseline):** median imputation, scaling, `C = 1`.
2. **Random forest:** 200 trees, unlimited depth. Train PR-AUC 1.000 but validation 0.644, so it overfits.
3. **Regularized random forest:** only `max_depth = 8` and `min_samples_leaf = 5` changed. Validation PR-AUC rose from 0.644 to 0.713, so this version was kept.
4. **Shallow Keras network:** 15 inputs, 16 ReLU units, 1 sigmoid output; Adam, batch 128, up to 60 epochs with early stopping on validation loss. The best epoch was 60, so a longer limit might help slightly.

Model comparison (threshold chosen on validation, then frozen)

Model	Split	ROC-AUC	PR-AUC	F1	Precision	Recall
Logistic regression	Validation	0.928	**0.716**	0.712	0.633	0.812

Model	Split	ROC-AUC	PR-AUC	F1	Precision	Recall
Logistic regression	Test	0.934	**0.712**	0.665	0.531	0.892
Random forest (regularized)	Validation	0.932	0.713	0.705	0.686	0.727
Random forest (regularized)	Test	0.940	0.707	0.667	0.628	0.711
Keras neural network	Validation	0.927	0.702	0.709	0.700	0.718
Keras neural network	Test	0.933	0.705	0.670	0.622	0.726

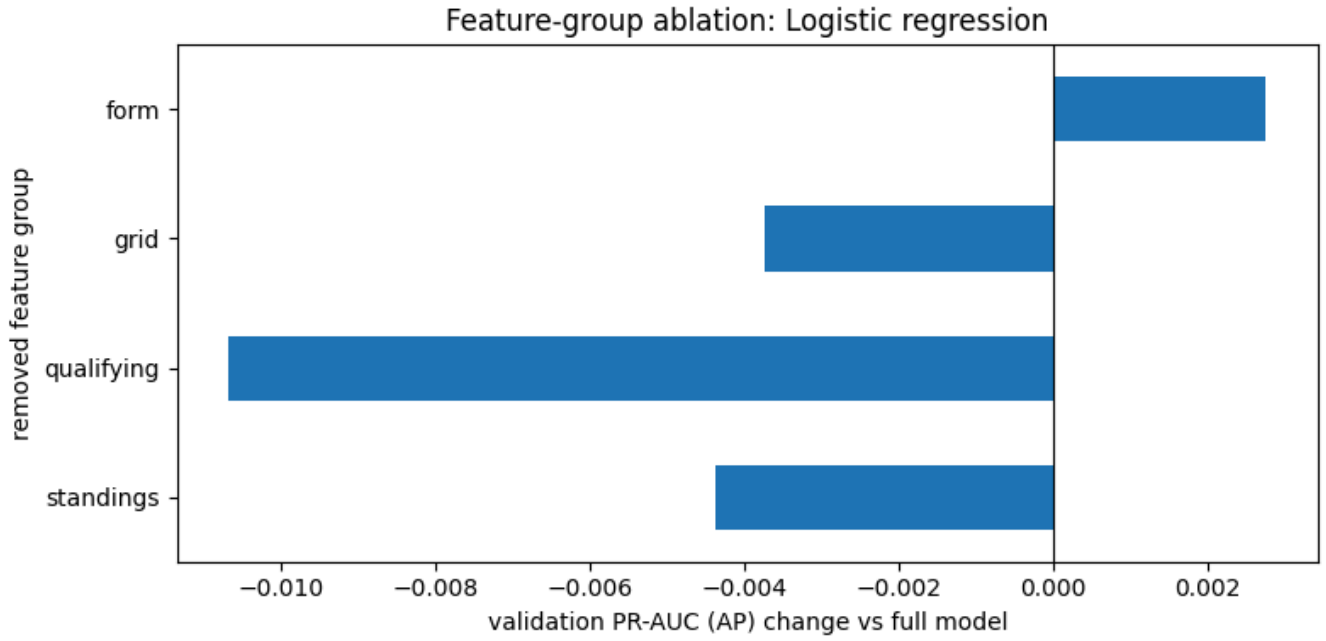
The final model is logistic regression (threshold 0.246), chosen on validation PR-AUC before looking at the test set. The test PR-AUC (0.712) is close to validation, so the model holds up in the new 2022 era. On test, the regularized forest has a higher ROC-AUC (0.940), and both the forest and the network have slightly higher F1 and clearly higher precision at their thresholds, while logistic regression has the highest recall. So the models trade precision against recall differently; on our primary metric they did not improve on the baseline. Train scores for all models are in the notebook (4.8). (Figure 8)



Feature-group ablation (remove one group, retrain logistic regression, validation PR-AUC)

Removed group	Features left	Validation PR-AUC	Change vs full
None (full model)	15	0.7157	0.0000
Qualifying	11	0.7050	-0.0107
Standings	11	0.7113	-0.0044
Grid	12	0.7119	-0.0037
Form	11	0.7184	+0.0028

Removing qualifying hurts the most. Removing grid hurts less than expected because `quali_pos` (with its grid fallback) and `grid_penalty` still carry grid information. Form provided no observed validation benefit in this ablation (removing it raised PR-AUC by 0.0028). This is a small difference and not a causal effect. (Figure 9)



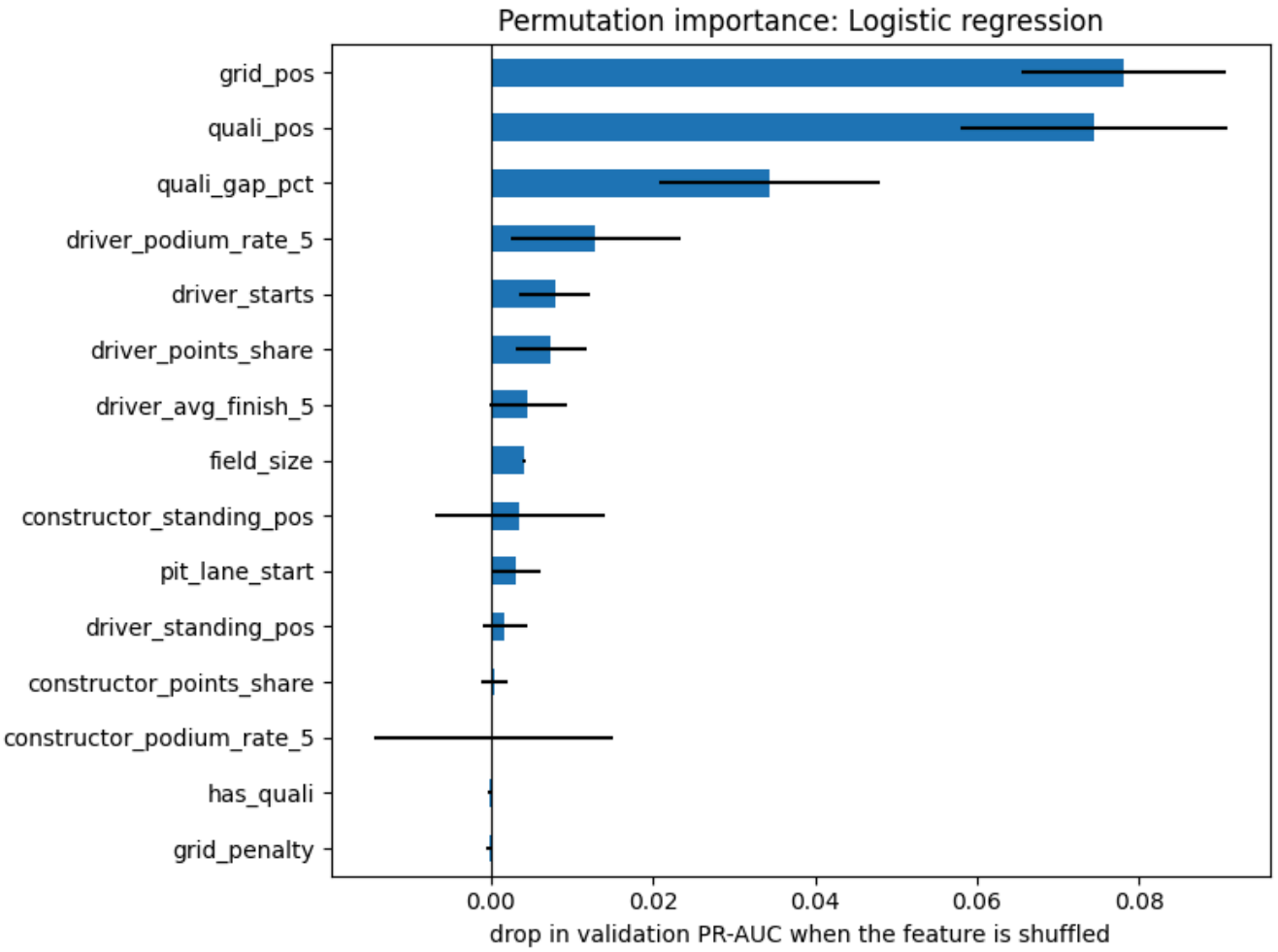
Inference function: `predict_podium(record, model, threshold)` takes one driver's pre-race inputs and returns a probability and a yes/no. On a complete validation record it gives 0.752 (podium). With qualifying and standings removed it still works (0.627), using the grid fallback and training medians.

6. Explainability (XAI)

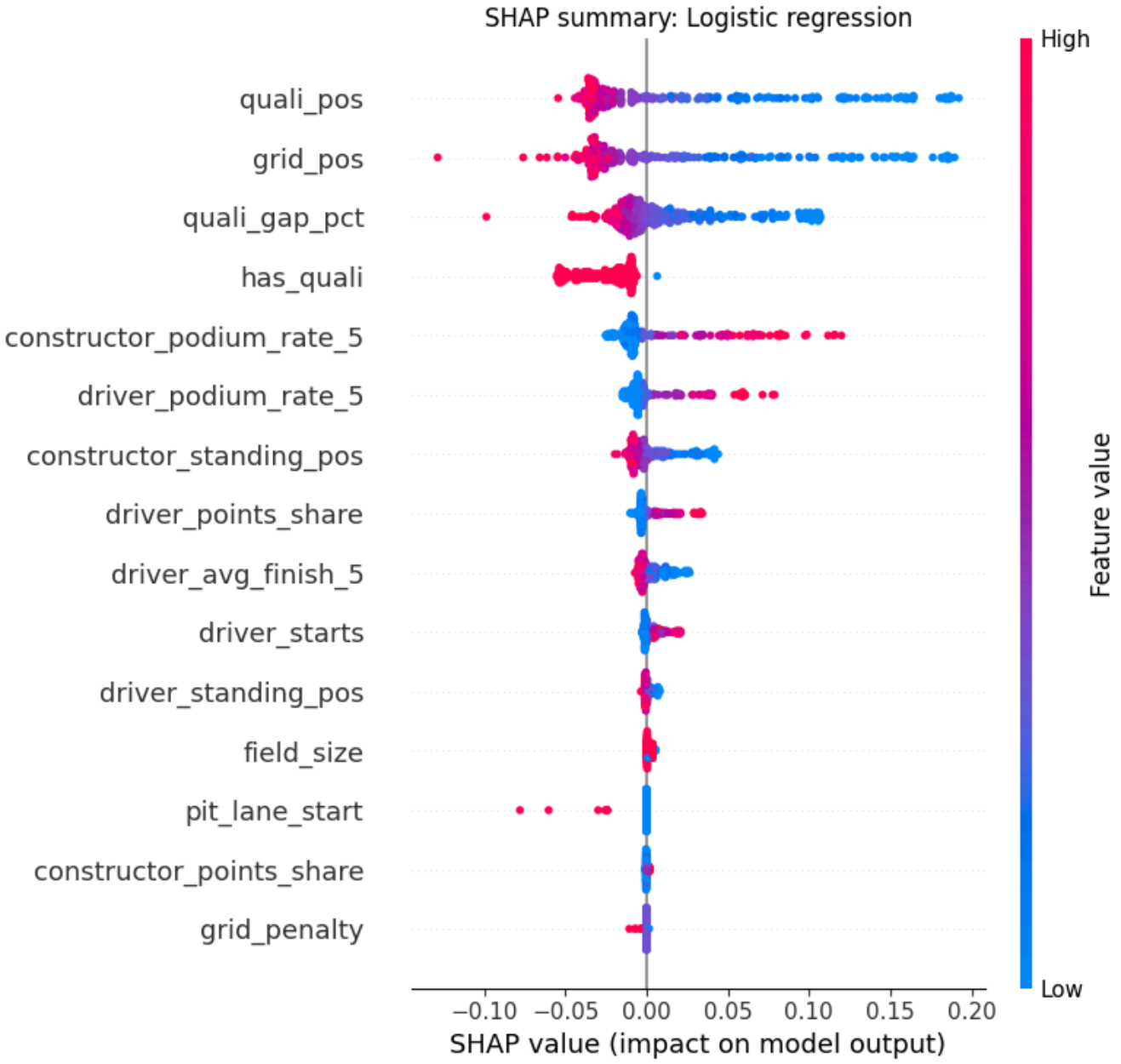
The model relies mostly on where the driver starts: grid position, qualifying position and the qualifying gap to pole are the top three features for both permutation importance and SHAP.

> **These explanations are not causal.** A feature being important only means the model uses it to make predictions. It does not prove that the feature causes podium finishes. Many features are linked (a fast car gives a good grid position, good form and good standings), so the real reason can be the car.

Global explanation 1: permutation importance. We shuffle one feature at a time on the validation seasons (10 repeats) and measure the drop in PR-AUC. Shuffling `grid_pos` costs 0.078, `quali_pos` 0.074 and `quali_gap_pct` 0.034. Form and standings features cost about 0.01 or less each, and `has_quali` and `grid_penalty` cost nothing. (Figure 10)

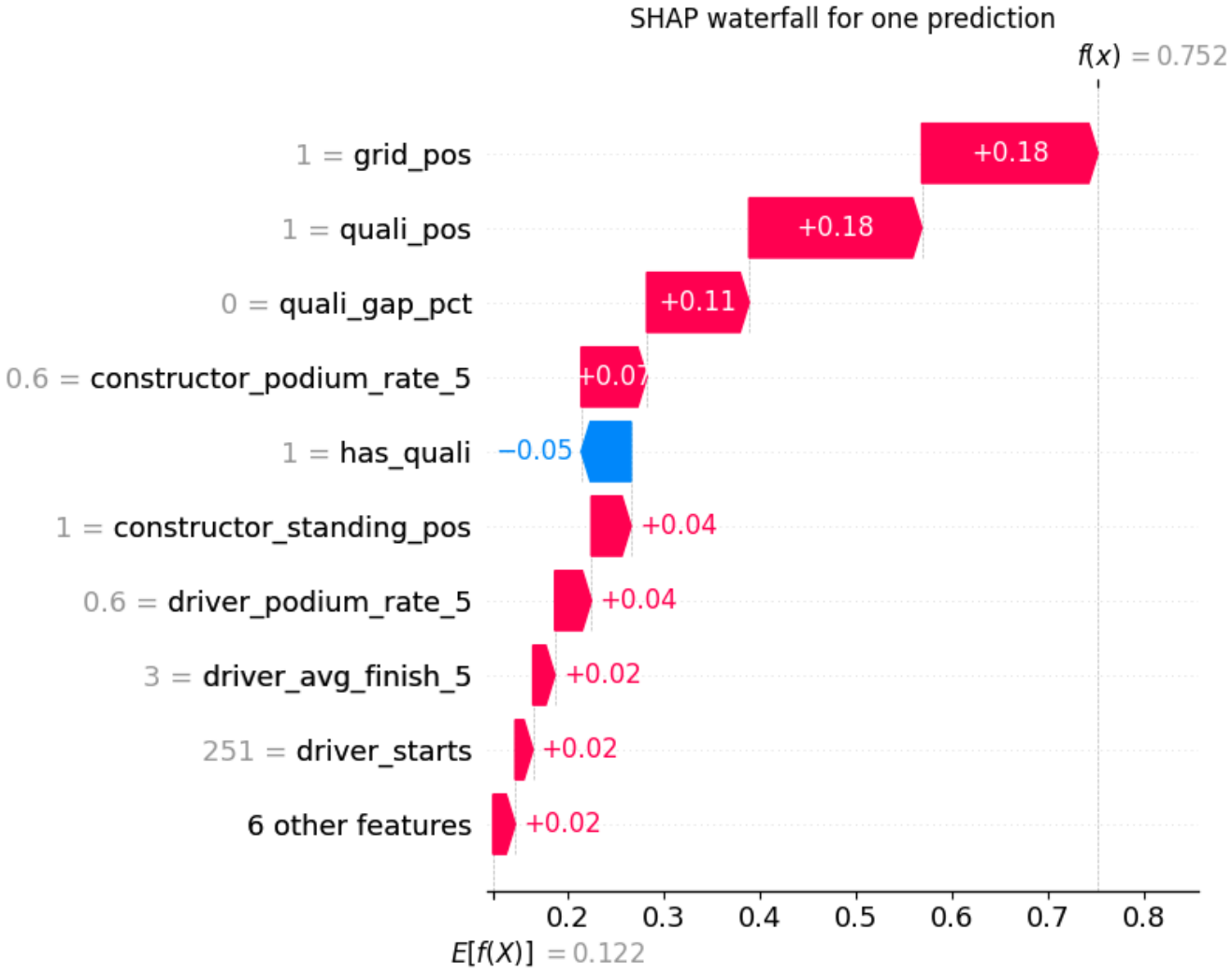


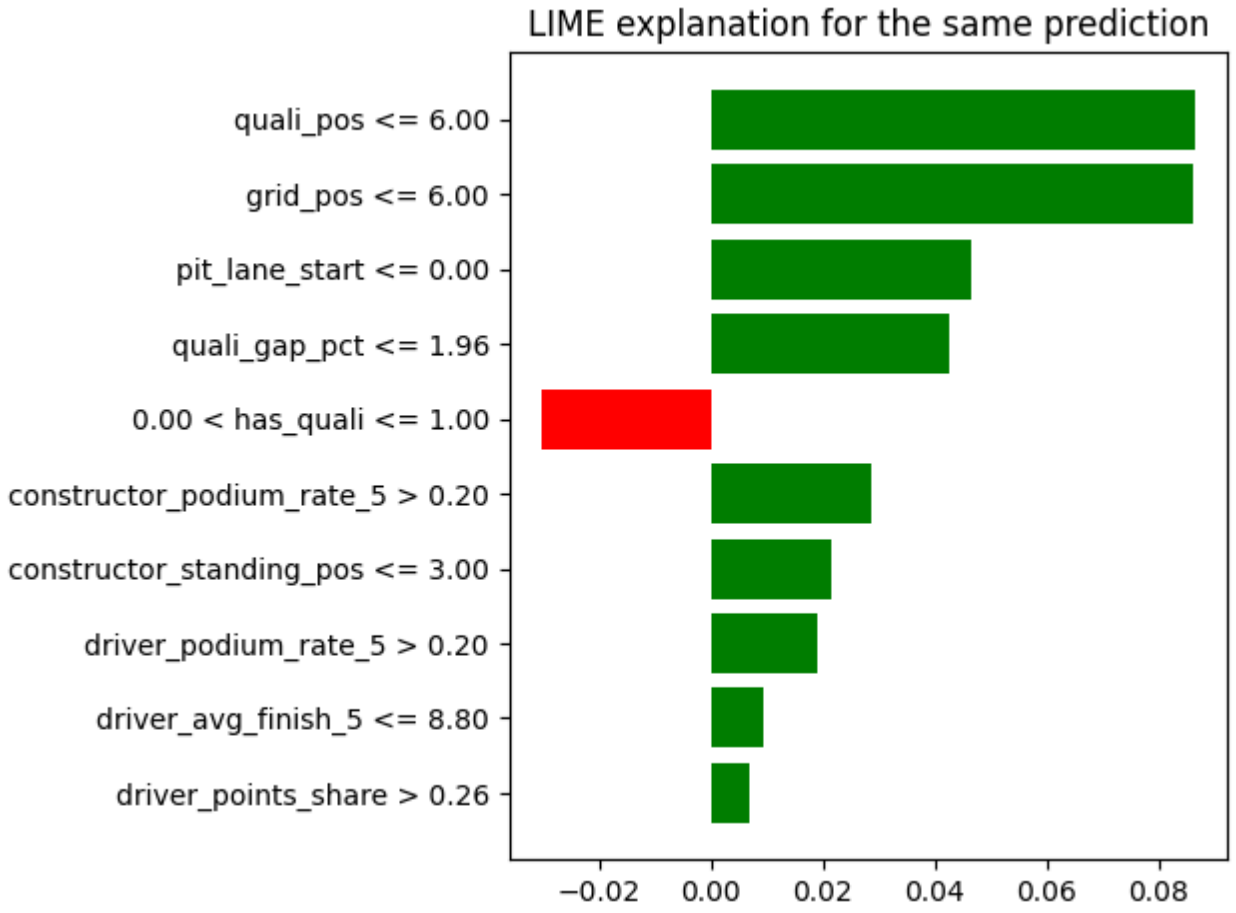
Global explanation 2: SHAP. On 300 validation drivers against 100 training drivers as background, the top three by mean |SHAP| are `quali_pos`, `grid_pos` and `quali_gap_pct`, the same three as permutation importance. `has_quali` ranks 4th in SHAP but 14th in permutation importance: almost every validation driver has qualifying data, so shuffling it changes very little, while SHAP compares each driver with training drivers who mostly had none. It reflects the difference between old and new seasons, not a reason for podiums. (Figure 11)



Local explanation: one prediction. Lewis Hamilton at the 2020 Styrian Grand Prix started from pole. The model gave him a 75 % podium probability against an average of 12 %, and he did finish on the podium.

- **SHAP waterfall:** starting 1st (+0.18), qualifying 1st (+0.18) and a qualifying gap of 0 (+0.11) raised the probability most; his team's recent podiums added +0.07. (Figure 12)
- **LIME** agrees: its two biggest reasons are `quali_pos <= 6` and `grid_pos <= 6` (+0.086 each). (Figure 13)


SHAP waterfall for one prediction
f(x) = 0.752
1 = grid_pos
+0.18
1 = quali_pos
+0.18
0 = quali_gap_pct
+0.11
0.6 = constructor_podium_rate_5
+0.07
1 = has_quali
−0.05
1 = constructor_standing_pos
+0.04
0.6 = driver_podium_rate_5
+0.04
3 = driver_avg_finish_5
+0.02
251 = driver_starts
+0.02
6 other features
+0.02
0.2
0.3
0.4
0.5
0.6
0.7
0.8
E[f(X)] = 0.122



Limits: credit is split unevenly between correlated features (grid and qualifying position are almost the same thing), SHAP uses samples to stay fast, and LIME varies slightly with its random sampling (we fix the seed).

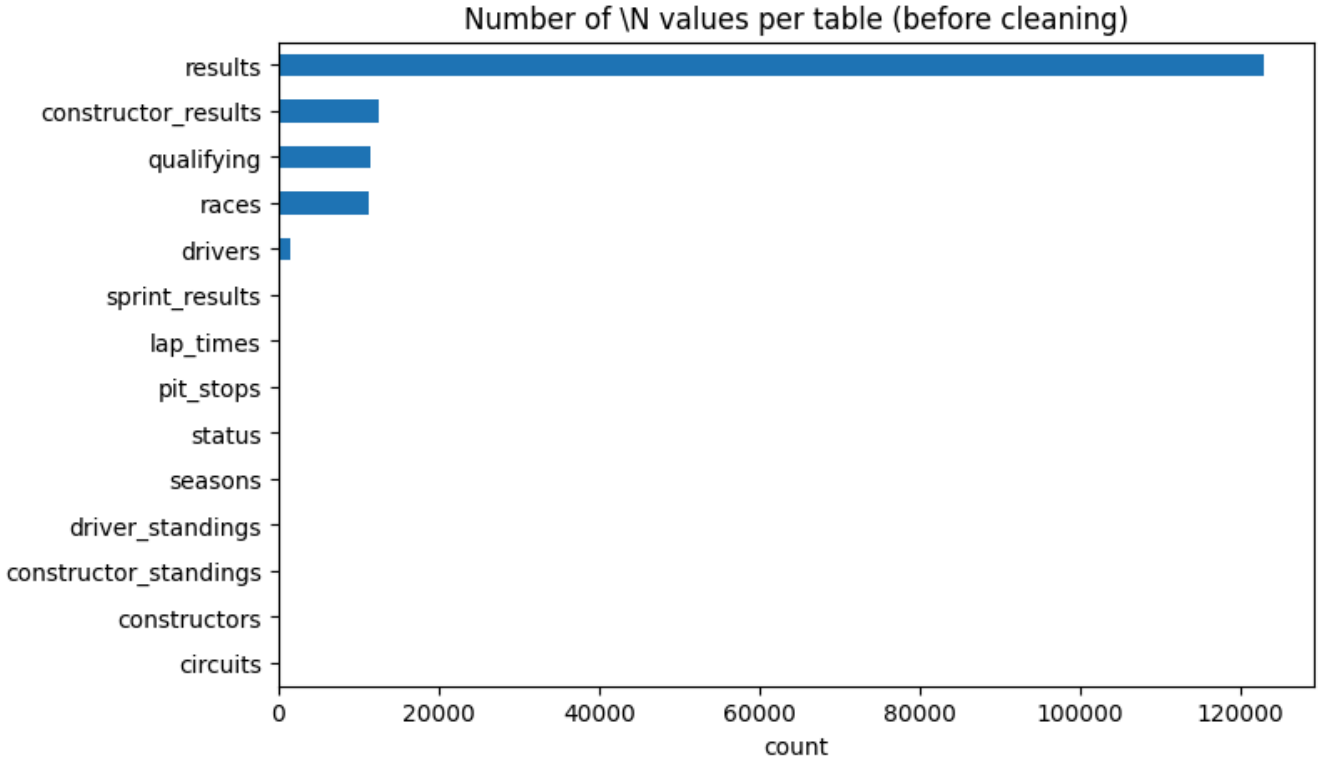
7. Limitations and conclusion

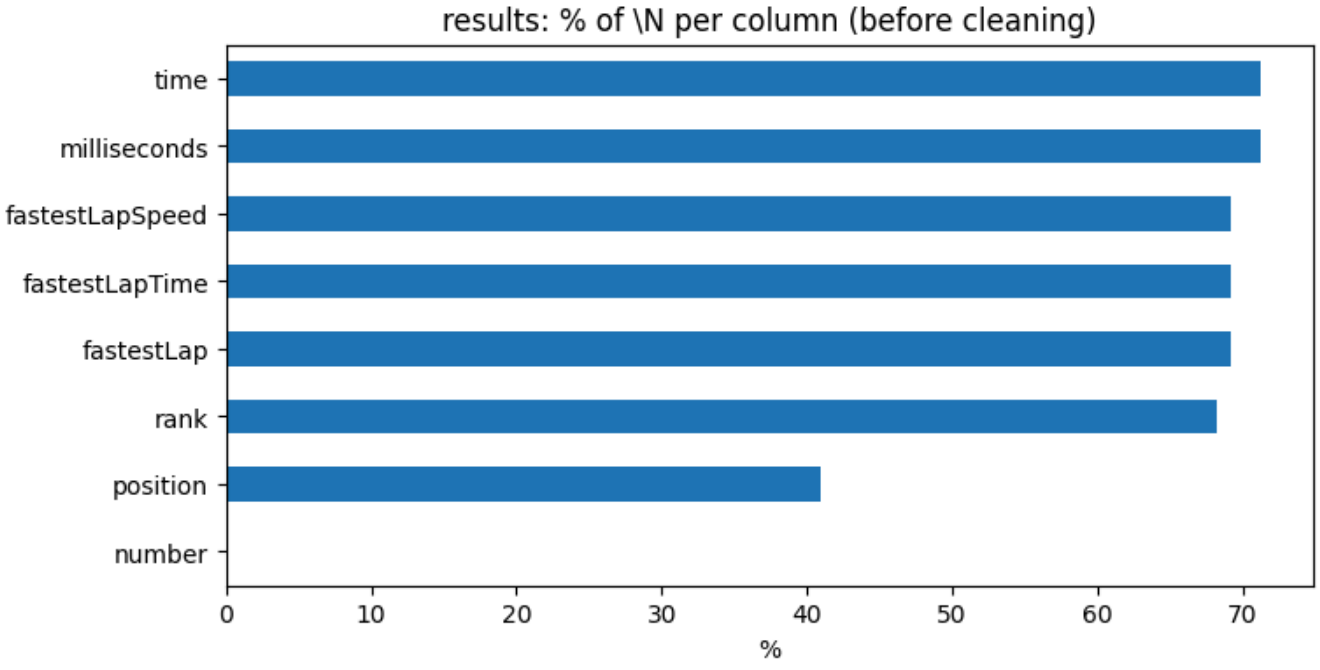
Known limitations

- **Qualifying-coverage drift:** there is no qualifying data before 1994 and it is patchy until 2002, so 63.7 % of training rows have no qualifying gap and rely on the grid fallback and median imputation.
- **Era sensitivity:** the oldest training seasons (bigger fields, worse reliability, different points systems) differ a lot from the 2022 to 2024 test era with its new car rules.
- **Team renames** reset constructor history (for example Toro Rosso to AlphaTauri to RB).
- **One seed and one validation window** do not show how much the scores would vary; the bootstrap noise on validation PR-AUC is about 0.056.
- **Correlated feature groups** limit how far the ablation and XAI results can be interpreted.
- **No per-race constraint:** the model can predict more or fewer than three podiums in a race.

Conclusion. A logistic regression on 15 pre-race features predicts podium finishes with a test PR-AUC of 0.712 and ROC-AUC of 0.934. Starting position and qualifying carry most of the signal and standings add a little, while form showed no observed validation benefit in the ablation. It was selected on validation PR-AUC; the random forest and shallow neural network offered different precision–recall tradeoffs rather than a better primary score.

Appendix A. Cleaning: before and after





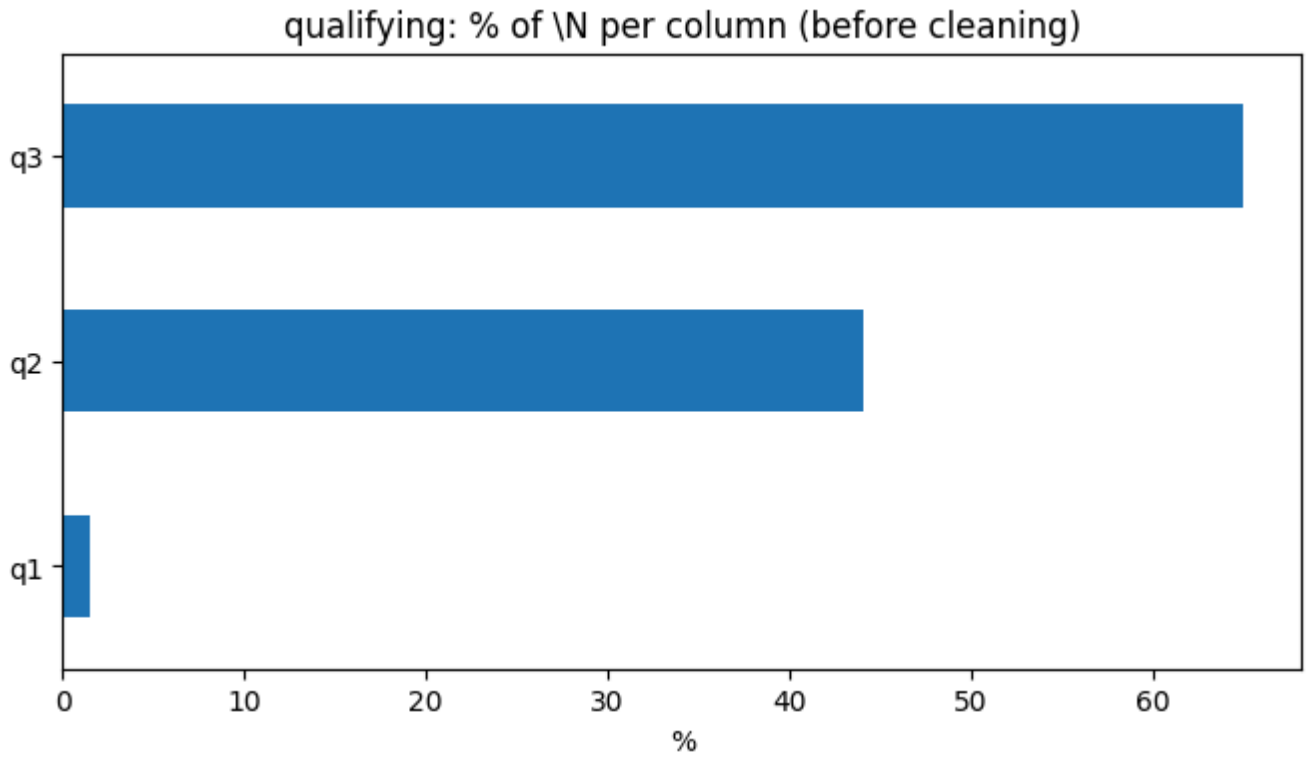


Table	Rows before	Rows after	\N before	NaN after	Number columns before	Number columns after
results	26,759	26,668	122,887	122,339	9	15
qualifying	10,494	10,494	11,600	11,668	6	9
races	1,125	1,125	11,349	11,349	4	4
constructor_results	12,625	12,625	12,608	12,608	4	4
drivers	861	861	1,559	1,559	1	2
sprint_results	360	360	73	73	10	13
lap_times	589,081	589,081	0	0	5	5
pit_stops	11,371	11,371	0	0	5	5
driver_standings	34,863	34,863	0	0	6	6
constructor_standings	13,391	13,391	0	0	6	6
circuits	77	77	0	0	4	4
constructors	212	212	0	0	1	1
seasons	75	75	0	0	1	1
status	139	139	0	0	1	1

Cleaning turned every \N into a real missing value, so the counts move from the \N column to the NaN column. Columns that only held numbers became numeric (results 9 to 15 numeric columns, qualifying 6 to 9, sprint_results 10 to 13). Results lost 91 duplicate rows,

which is why its missing count drops slightly. Qualifying has 68 more missing values after cleaning because some of its cells were already empty in the CSV instead of holding \N, so they were not counted in the \N column.

Appendix B. Feature evidence and preprocessing

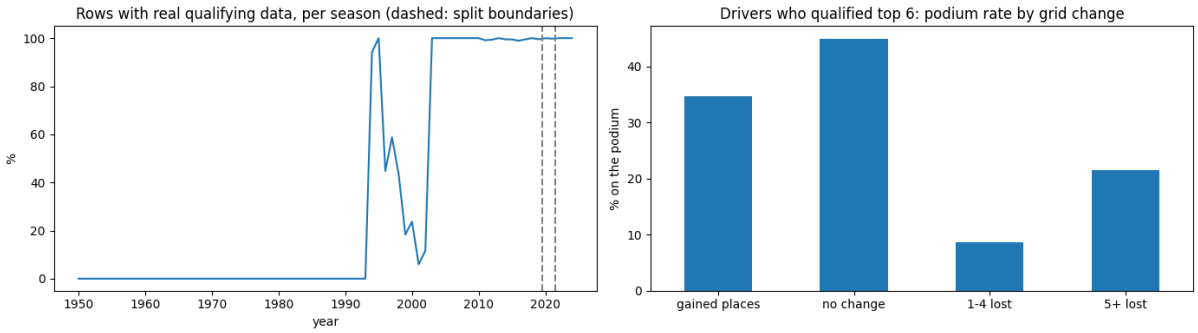


Figure B1: Share of each season with real qualifying data, and podium rate by places lost between qualifying and the grid (training seasons)

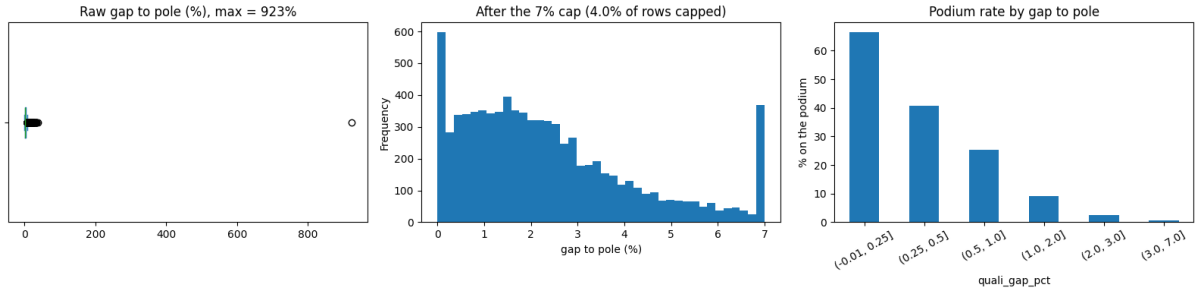


Figure B2: Qualifying gap to pole before and after the 7 % cap

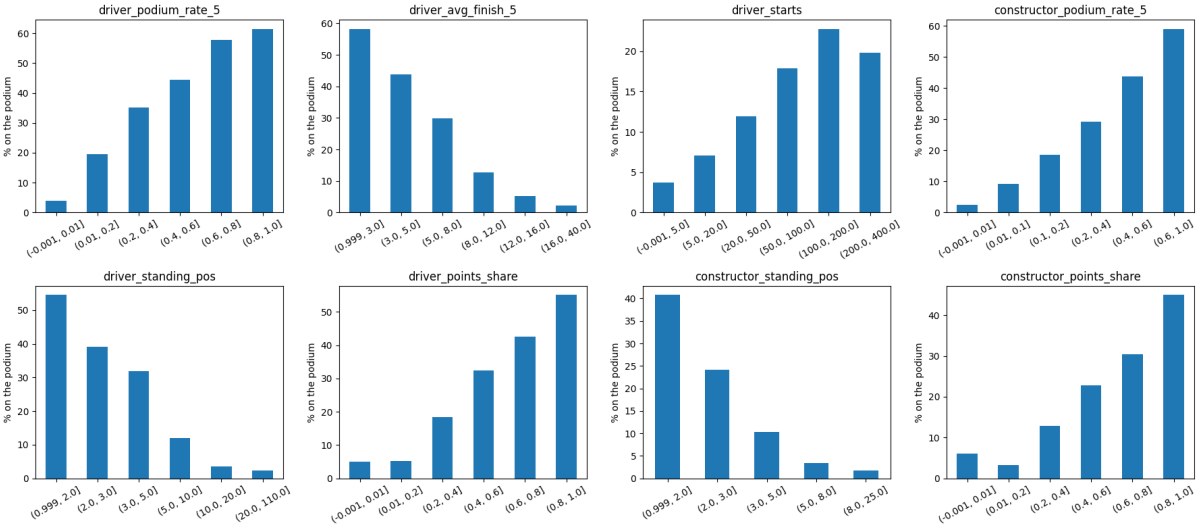


Figure B3: Podium rate by bins of each form and standings feature (training seasons)

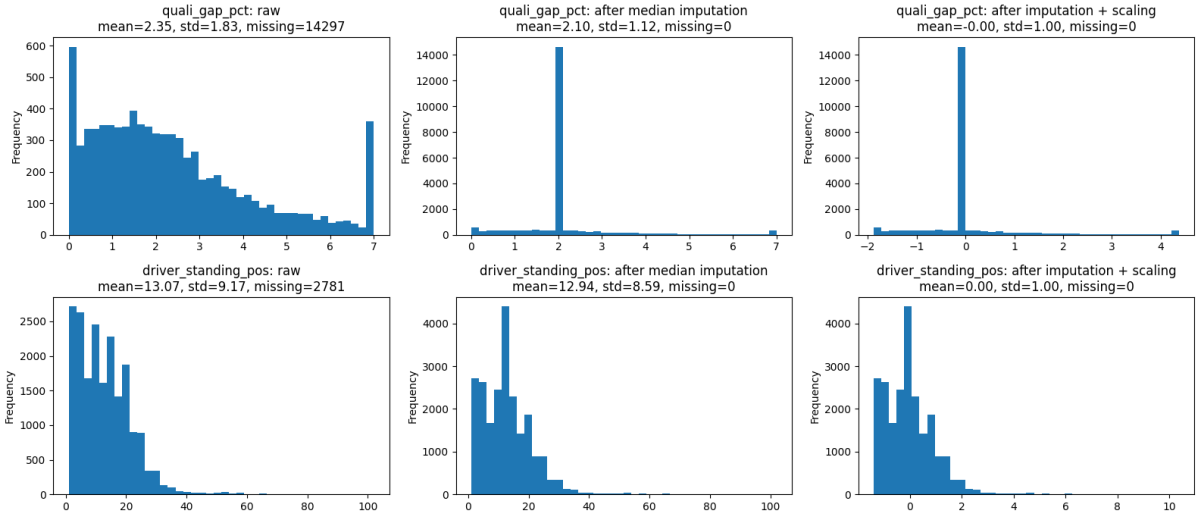


Figure B4: Feature distributions raw, after median imputation, and after scaling

Appendix C. Model diagnostics



Figure C1: Train and validation PR-AUC for each attempt

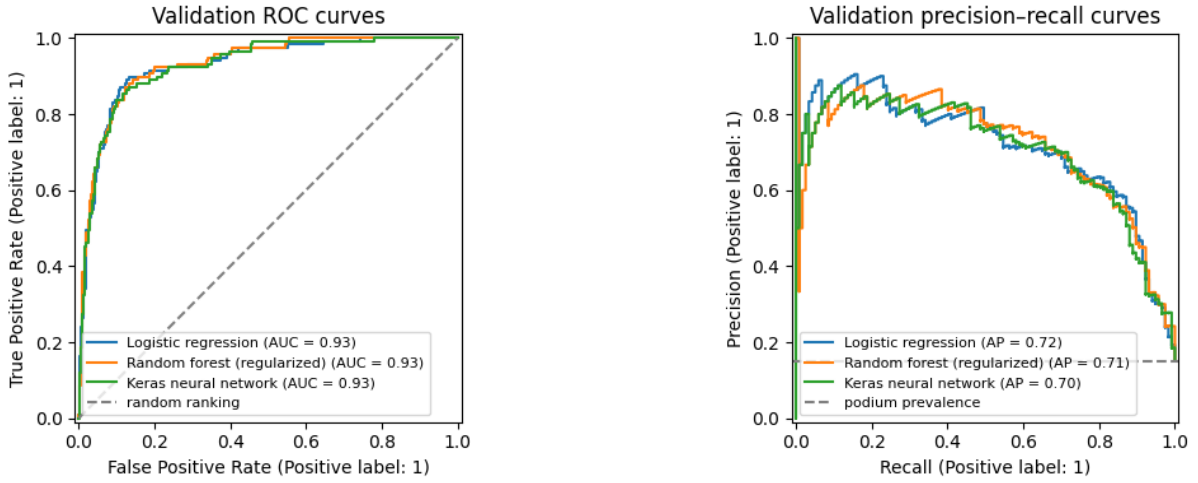


Figure C2: Validation ROC and precision–recall curves

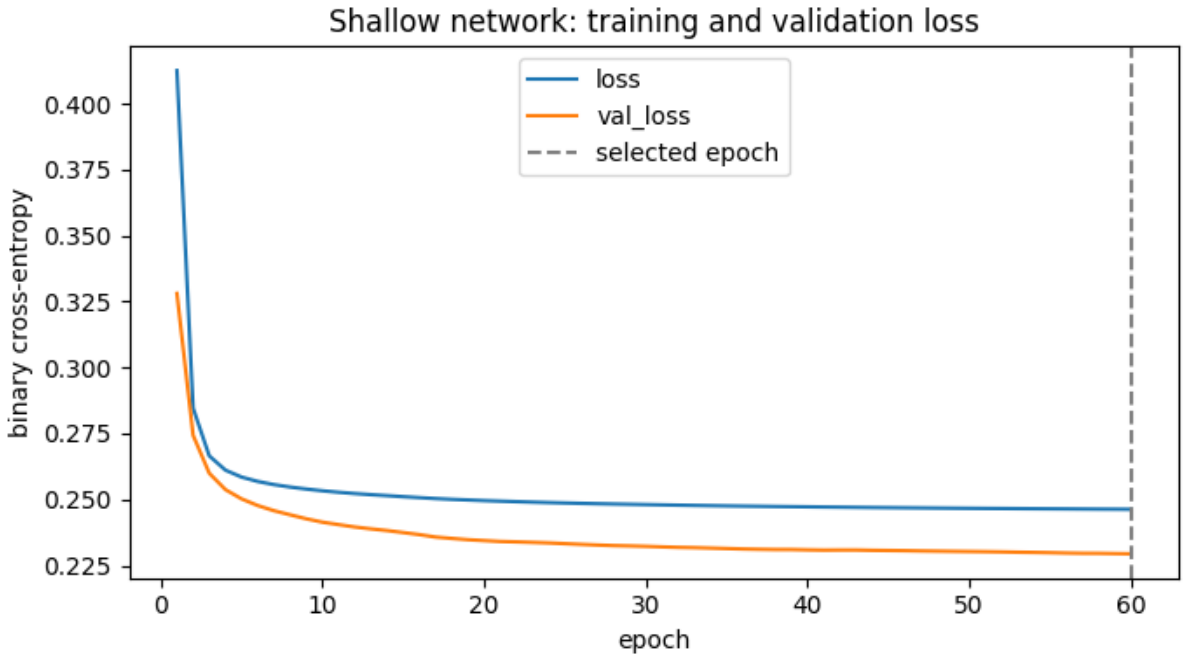


Figure C3: Keras network learning curve (validation loss still falling at epoch 60)